How I Used the Devpost Learn Skills

A short account of the planning process behind **Devil's Rock Paper Scissor**

Written from my perspective as the game's creator: I used the Devpost Learn skills to turn my existing idea into a reviewed plan before building. The challenge asks participants to use the skills and include **scope.md**, **prd.md**, and **spec.md** in the repository. Read the challenge brief. The sections below explain what each stage contributed.

The process

SKILL	WHAT WE WORKED THROUGH	RESULT
1-start	I described the game, my development background, what I wanted to learn, and how I wanted to collaborate. This helped the later interviews match my experience.	A private learner profile for the skills to carry forward; it is not part of the public submission.
2-scope	We shaped my initial game description into its audience, distinctive idea, player loop, submission boundary, and exclusions. When I added bots, leaderboards, and quests, we revisited the boundary instead of silently treating them as out of scope.	**scope.md** Approved product goal and proof-of-concept boundary.
3-prd	We walked through the player journey and clarified screens, rules, feedback, defaults, failure cases, and what would count as working. Timed card phases, claim-versus-action, matchmaking fallback, and the result reveal became specific enough to implement and review.	**prd.md** Approved player-facing behavior and acceptance criteria.
4-spec	We mapped those behaviors to system responsibilities, data ownership, interfaces, and the web target. I asked for technical recommendations where I wanted guidance, then agreed on the consequential choices.	**spec.md** Approved technical blueprint.

How the interview worked

The interviews stayed focused: usually one to three related questions at a time. I supplied the product decisions; Codex, the AI collaborator, organized them, surfaced contradictions or consequential gaps, and kept unresolved choices visible. Codex drafted documents once there was enough information, showed them for my review, and changed their status only after I explicitly approved them. The scope was approved on October 20, the revised scope and PRD on October 21, and the spec on October 22, 2026.

> **A clarification in practice.** I clarified that every newly appearing quest starts at zero and counts only later play. The PRD and spec were updated to replace the earlier interpretation. This kept a tentative reading from becoming a hidden product decision.

Technical choices and ownership

The spec records the agreed division of work: Godot coordinates a live match, GD-Sync carries player-to-player match activity, and a TypeScript API on Vercel uses MongoDB for saved profiles, results, statistics, and quest progress. It also explains the trusted-host tradeoff and the limits of validating history submitted by a client. These are planned responsibilities; approval of the spec does not by itself prove service compatibility or runtime behavior.

Codex helped compare and explain technical approaches, keep the documents consistent, and translate my decisions into implementation-ready requirements. I retained ownership of the game rules, scope, and consequential technical choices. My project instructions added Godot and game-jam constraints: readable code, .tres-driven tuning, single-threaded web compatibility, and asking rather than guessing when a decision was unclear. Those constraints are captured in **docs/AI_Bootstrap.md**.

From plan to implementation

After the spec, I chose a flow-first build order. I used **5-build** in the planning chat to turn that order into an approved 12-milestone plan and matching checklist, with implementation assigned to separate milestone chats. The original plan was later adjusted to stage final integration and add stability and polish work. The current checklist records those changes.

The milestone reports distinguish development simulations from real networking and persistence, and record the checks actually performed. The planning documents define what the project is meant to do; the reports provide evidence about what was implemented and observed.

Project documents

devpost/scope.md | **devpost/prd.md** | **devpost/spec.md**
docs/Milestones.md | **devpost/checklist.md** | **docs/reports/**

> **Plan versus evidence.** A reviewed plan is not presented as proof that every planned behavior or service was verified. The plan and implementation reports are separate so readers can see both what I intended to build and what was actually observed.

Build With AI: Basics on Devpost